

当然！将深度学习项目封装为接口并集成到系统，同时设计一个合理的数据库，这是一个非常经典且重要的工业AI项目流程。我将为您提供一个从数据库设计到API封装的完整方案。

第一部分：数据库设计 (Database Design)

数据库的核心任务是：**存储原始数据、存储检测结果、管理模型版本、以及用户和系统操作日志**。建议使用关系型数据库（如 MySQL, PostgreSQL）结合文件存储系统（如本地存储、MinIO、AWS S3等）。

1. 核心数据表设计

以下是几个关键的表：

a. 表：welding_images（焊缝图像表） 存储所有待检测和已检测的原始图像。

字段名	类型	说明
id	INT (PK, AI)	主键，自增ID
original_image_path	VARCHAR(255)	原始图像在文件服务器上的路径 (非常重要，避免直接存二进制文件到DB)
upload_time	DATETIME	图像上传时间
upload_by	VARCHAR(100)	上传者（可以是用户或系统名称）
related_project	VARCHAR(100)	关联的项目或工单号
image_meta	JSON	图像元信息（可选），如：相机型号、拍摄时间、焊缝编号等

b. 表：detection_results（检测结果表） 存储每次检测的结果，与 welding_images 是一对多的关系（一张图可能被不同模型多次检测）。

字段名	类型	说明
id	INT (PK, AI)	主键，自增ID
image_id	INT (FK)	外键 ，关联 welding_images.id
model_id	INT (FK)	外键 ，关联 models.id (记录使用的是哪个模型版本)
detection_time	DATETIME	检测完成时间
processing_time	FLOAT	模型推理耗时（单位：秒）

字段名	类型	说明
is_defective	BOOLEAN	总体结论： 是否有缺陷 (True/False)
confidence	FLOAT	整体置信度或最大缺陷的置信度
result_json_path	VARCHAR(255)	详细结果文件路径 (JSON格式, 包含所有缺陷框、类别、置信度)
overlay_image_path	VARCHAR(255)	可视化结果图路径 (将缺陷框画在原图上的图片, 用于前端展示)
status	VARCHAR(20)	检测状态: pending, processing, completed, failed

c. 表: defect_details (缺陷详情表) 存储一张图片中每一个具体的缺陷实例, 与 detection_results 是一对多的关系 (一个检测结果可能包含多个缺陷)。

字段名	类型	说明
id	INT (PK, AI)	主键
result_id	INT (FK)	外键 , 关联 detection_results.id
defect_type	VARCHAR(50)	缺陷类别 (如: 气孔porosity, 裂纹crack)
confidence	FLOAT	该缺陷的置信度
bbox_coords	JSON	缺陷边界框坐标 [x_min, y_min, x_max, y_max]
segment_coords	JSON	(如果是实例分割) 缺陷的多边形轮廓点坐标

d. 表: models (模型管理表) 管理不同的模型版本, 这对于模型迭代和A/B测试至关重要。

字段名	类型	说明
id	INT (PK, AI)	主键
model_name	VARCHAR(100)	模型名称 (如: yolov8s-weld-v2.1)
model_path	VARCHAR(255)	模型权重文件 (.pt或.onnx) 的路径
model_class_path	VARCHAR(255)	类别标签文件 (classes.txt) 的路径
version	VARCHAR(50)	版本号
is_active	BOOLEAN	是否是当前激活使用的模型
upload_time	DATETIME	模型上传时间
description	TEXT	模型描述、性能指标 (mAP, Precision等)

e. 表: users / system_logs (用户与日志表) (根据系统复杂度可选) 用于权限管理和审计。

2. 文件存储方案 (File Storage)

- 原始图片、结果JSON、可视化图片 和 模型文件 都应该存储在数据库之外。
- 推荐使用对象存储 (Object Storage) :
 - MinIO: 开源、S3兼容，可以自建私有云存储。**非常适合工业环境。**
 - AWS S3 / Azure Blob / Google Cloud Storage: 公有云方案，方便但可能需要网络开销。
- 不推荐直接将文件以BLOB形式存入数据库，会严重影响性能。

目录结构示例：

```
data-bucket/
├── original_images/
│   ├── project_a/
│   │   ├── 2024-01-01/
│   │   │   └── image_001.jpg
│   │   ├── 2024-01-02/
│   │   │   └── image_002.jpg
│   └── project_b/
├── results_json/
│   └── 10086.json # 结果ID为10086的JSON文件
├── overlay_images/
│   └── 10086.jpg # 结果ID为10086的可视化图片
└── models/
    ├── yolov8s-weld-v2.1/
    │   ├── model.onnx
    │   └── classes.txt
    ├── unet-seg-v1.0/
    │   └── ...
```

第二部分：接口封装 (API Integration)

使用一个高效的Web框架来封装你的深度学习模型。**FastAPI** 是当前的最佳选择，因为它性能极高（基于ASGI），自带自动交互式API文档，并且非常易于使用。

1. 接口设计 (RESTful API)

a. 健康检查端点

```
GET /health
```

响应: {"status": "ok", "model": "yolov8s-weld-v2.1"}

b. 模型预测端点 (核心)

```
POST /api/predict
```

请求体 (form-data):

- file: 上传的图像文件 (必选)
- model_version: (可选) 指定使用的模型版本, 不指定则使用激活模型
- confidence_threshold: (可选) 置信度阈值

响应 (JSON):

```
{
  "result_id": 10086,
  "is_defective": true,
  "defects": [
    {
      "defect_type": "crack",
      "confidence": 0.95,
      "bbox": [100, 150, 200, 250]
    },
    {
      "defect_type": "porosity",
      "confidence": 0.87,
      "bbox": [300, 400, 320, 420]
    }
  ],
  "processing_time": 0.45,
  "overlay_image_url": "/api/results/10086/overlay"
}
```

c. 结果查询端点

```
GET /api/results/{result_id}
```

响应: 返回存储在数据库中的完整结果信息。

d. 历史查询端点

```
GET /api/history?project=project_a&page=1&size=20
```

响应: 分页返回某个项目下的所有检测历史记录。

2. FastAPI 服务核心代码示例

```
from fastapi import FastAPI, File, UploadFile, BackgroundTasks
from fastapi.responses import JSONResponse
from pydantic import BaseModel
from typing import List, Optional
import cv2
import numpy as np
import json
```

```

import time
from your_project import predict # 你的深度学习模型推理函数
from database import SessionLocal, WeldingImage, DetectionResult, DefectDetail # 假设的数据库模型

app = FastAPI(title="Weld Defect Detection API")

# 1. 依赖项：获取数据库会话
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# 2. 定义Pydantic模型（用于请求/响应校验）
class DefectOut(BaseModel):
    defect_type: str
    confidence: float
    bbox: List[int]

    class Config:
        orm_mode = True

class PredictionResult(BaseModel):
    result_id: int
    is_defective: bool
    defects: List[DefectOut]
    processing_time: float
    overlay_image_url: str

# 3. 核心预测端点
@app.post("/api/predict", response_model=PredictionResult)
async def create_prediction(
    background_tasks: BackgroundTasks,
    file: UploadFile = File(...),
    db: Session = Depends(get_db)
):
    # 读取上传的图片
    contents = await file.read()
    nparr = np.frombuffer(contents, np.uint8)
    image = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    # 将图像信息存入数据库，状态为 `pending`
    db_image = WeldingImage(original_image_path=f"temp/{file.filename}",
upload_time=datetime.now())
    db.add(db_image)
    db.commit()
    db.refresh(db_image)

    # 创建一条空的检测记录
    db_result = DetectionResult(
        image_id=db_image.id,
        detection_time=datetime.now(),
        status="processing"
    )
    db.add(db_result)

```

```

db.commit()
db.refresh(db_result)

# 将实际推理任务放入后台, 立即返回result_id, 实现异步处理
background_tasks.add_task(run_detection, image, db_result.id, file.filename, db)

return JsonResponse(
    content={
        "result_id": db_result.id,
        "status": "processing",
        "message": "Detection started. Please query the result later with the result_id."
    }
)

# 4. 后台推理任务
def run_detection(image, result_id, filename, db):
    try:
        start_time = time.time()
        # 调用你的模型进行预测
        detection_results = predict(image) # 你的模型预测函数, 返回缺陷列表等信息
        end_time = time.time()

        processing_time = round(end_time - start_time, 2)

        # 解析结果, 存入数据库
        # ... (代码省略: 将结果保存到 detection_results 和 defect_details 表) ...

        # 更新检测结果状态为 `completed`
        db_result = db.query(DetectionResult).filter(DetectionResult.id == result_id).first()
        db_result.status = "completed"
        db_result.processing_time = processing_time
        db_result.is_defective = len(detection_results["defects"]) > 0
        # ... 设置其他字段 ...
        db.commit()

    except Exception as e:
        # 更新状态为 `failed`
        db_result = db.query(DetectionResult).filter(DetectionResult.id == result_id).first()
        db_result.status = "failed"
        db.commit()
        print(f"Detection failed for result_id {result_id}: {e}")

# 5. 结果查询端点
@app.get("/api/results/{result_id}", response_model=PredictionResult)
async def get_result(result_id: int, db: Session = Depends(get_db)):
    db_result = db.query(DetectionResult).filter(DetectionResult.id == result_id).first()
    if not db_result:
        return JsonResponse(status_code=404, detail="Result not found")
    return db_result

```

第三部分：系统集成与 workflows (System Integration & Workflow)

1. 前端/客户端:

- 工人或质检员通过Web界面或桌面客户端上传焊缝图片。
- 客户端调用 `POST /api/predict`，并立即得到 `result_id`。
- 客户端可以轮询 `GET /api/results/{result_id}` 来获取检测进度和最终结果。

2. 后端服务 (FastAPI):

- 接收请求，将任务异步化处理，快速响应，避免阻塞。
- 调用深度学习模型进行推理。
- 将原始数据、结果、日志持久化到数据库和文件存储 (MinIO)。

3. 深度学习模型:

- 被封装为一个独立的Python模块 (`your_project.predict`)。
- 最好使用固定的输入输出格式，例如输入是NumPy数组(CV2图像)，输出是一个包含缺陷列表、置信度、坐标的字典。

总结与建议

- **异步处理是关键**：模型推理可能很耗时，一定要用后台任务 (Celery, BackgroundTasks) 处理，不要让HTTP请求一直等待。
- **数据库关系要清晰**：遵循上述表结构，可以高效地追溯历史、分析数据、管理模型。
- **文件与数据库分离**：使用MinIO等对象存储来管理文件，数据库只存路径。
- **考虑未来扩展**：
 - **主动学习**：可以增加一个 `is_reviewed` 字段，让专家复核模型结果，并将复核后的数据自动加入训练集。
 - **模型监控**：记录每次预测的输入和输出，用于监控模型在生产环境中的性能漂移 (Data Drift)。
 - **Docker化**：将FastAPI应用和模型依赖打包成Docker容器，便于部署和扩展。

这个方案提供了一个坚实且可扩展的基础，您可以根据项目的具体需求进行调整和深化。